Автономная некоммерческая организация высшего образования
«Университет Иннополис»

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
(БАКАЛАВРСКАЯ РАБОТА)
по направлению подготовки
09.03.01 «Информатика и вычислительная техника»

FINAL QUALIFICATION WORK
(BACHELOR'S THESIS)
Academic Program
09.03.01 «Computer Science»

Направленность (профиль) образовательной программы
«Анализ данных и искусственный интеллект»

Field of Study:
«Data Science and Artificial Intelligence»

Тема	**Использование памяти и графов знаний для оценки действий интеллектуальных агентов**
Topic	**Memory-Assisted "LLM as a Judge" via Knowledge Graphs for Evaluating Agents**

Работу выполнил / *Prepared by*	**Пател Кхуш Прадипбхаи / Patel Khush Pradipbhai**	подпись / signature
Руководитель выпускной квалификационной работы / *Final Qualification Work Supervisor*	**Бадер Рашид / Bader Rasheed**	подпись / signature

Иннополис, Innopolis, 2026

Content

1. Introduction.....6

1.1. Relevance of the Topic..... 6

1.2. Problem Statement.....7

1.3. Purpose and Research Question..... 7

1.4. Objectives.....8

1.5. Object and Subject of Research.....8

1.6. Scientific Novelty.....9

1.7. Practical Significance..... 10

1.8. Structure of the Thesis.....10

2. Literature Review..... 11

2.1. LLM-as-a-Judge Evaluation..... 11

2.2. Retrieval-Augmented and Memory-Augmented Systems.....12

2.3. Monte Carlo Tree Search and Test-Time Reasoning.....12

2.4. Evaluation Validity, Leakage and Contamination..... 13

2.5. Statistical Methods for Evaluation..... 14

2.6. Summary.....14

3. Methodology.....16

3.1. Overview..... 16

3.2. System Architecture..... 16

3.2.1. The Typed Memory Graph.....17

3.2.2. The Memory-Assisted Judge.....17

3.2.3. Exploratory MCTS Components.....17

3.2.4. Evaluation Modes.....18

3.3. The Leakage-Free Evaluation Protocol..... 18

3.3.1. The Leakage Problem....18
3.3.2. Question-Level Partition.... 18
3.3.3. Frozen Memory During Evaluation.... 19
3.4. The Frozen-Memory Audit....19
3.4.1. Layer One: Snapshot and Fingerprint.... 19
3.4.2. Layer Two: Runtime Write-Blocking....19
3.4.3. Value of the Audit....20
3.5. Memory Conditions....20
3.6. Statistical Methodology....20
3.7. Limitations of the Methodology....21
3.8. Ethical Considerations....21
4. Implementation....22
4.1. Codebase....22
4.2. The Memory-Assisted Judge....22
4.3. The MCTS Components....23
4.4. The Frozen-Memory Audit....24
4.5. Structured Outputs....25
4.6. Handling Transient Failures.... 26
4.7. Reproduction Pipeline.... 27
5. Evaluation and Discussion.... 28
5.1. Dataset.... 28
5.2. Experimental Setup.... 29
5.3. When Memory Appears to Help: The Conventional Protocol.... 29
5.4. When Memory Fails to Help: The Leakage-Free Results.... 30
5.4.1. Clean-Memory Comparison.... 30
5.4.2. Self-Written versus Oracle Memory....34

5.4.3. Structure Ablation..... 34
5.4.4. Cross-Seed Stability..... 35
5.5. When Memory Appears to Corrupt: Poisoning and the Artifact.....36
5.6. Reliability Harness..... 38
5.7. Discussion: Why Memory Neither Helped nor Harmed..... 38
Why the apparent benefit disappeared.....39
Why memory carries no usable signal here.....39
Why the apparent collapse disappeared..... 39
When memory would be expected to help..... 40
5.8. Cost of the MCTS Modes.....40
5.9. Threats to Validity..... 41
6. Conclusion..... 42
6.1. Summary of the Work.....42
6.2. Answer to the Research Question.....42
6.3. Contributions..... 43
6.4. Future Work.....44
6.5. Concluding Remark.....44
Appendix A. Frozen-Memory Audit Record Format.....45
Appendix B. Evaluation Modes and Memory Conditions..... 46
Appendix C. Full Per-Mode Results..... 47
Appendix D. Reproduction..... 48
Bibliography cited.....49

Abstract

Large Language Models are increasingly used as automatic judges that score the outputs of other models. This thesis investigates whether equipping an LLM judge with a persistent, structured memory of its past evaluations improves its verdicts. A memory-augmented judging system, MCTS-MAJ, was implemented, together with a leakage-free evaluation protocol and a two-layer frozen-memory audit that proves memory is not modified during evaluation. Under a conventional protocol, memory appeared to deliver large gains; under the leakage-free, audited protocol no configuration significantly outperformed a single-pass stateless judge. This null result is established for a single judge model, GPT-4o, on an eighty-sample held-out set, and is therefore stated as 'no effect was detected at this scale' rather than as a universal claim. The audit also caught a real leakage defect, and a corrected re-run showed that a previously reported collapse under memory poisoning was a measurement artifact. The thesis explains, mechanistically, why memory neither helped nor harmed on this benchmark, and identifies the conditions under which it would be expected to help. Both the apparent benefit and the apparent fragility of memory are largely artifacts of the evaluation protocol.

Keywords: LLM-as-a-Judge, memory-augmented evaluation, retrieval-augmented generation, Monte Carlo Tree Search, evaluation leakage, frozen-memory audit, statistical significance, negative result.

Chapter 1

1. Introduction

1.1. Relevance of the Topic

Large Language Models (LLMs) are now used not only to generate text but also to *evaluate* it. In the LLM-as-a-Judge paradigm, a language model reads a task, a candidate response, and a grading rubric, and returns a verdict [1], [2], [3]. Automatic judges rank chatbots, score the outputs of automated agents, and provide reward signals during model training. Because their verdicts increasingly drive downstream decisions, the trustworthiness of an automatic judge is a first-order research concern [4], [5].

A standard LLM judge is *stateless*: it evaluates each item in isolation and retains nothing between evaluations. It cannot recognize that it has already seen a near-identical case, cannot reuse a correct line of reasoning, and cannot learn from a past mistake. This limitation motivates *memory-augmented* judging, in which the judge is connected to a persistent store of its past evaluations and retrieves relevant prior cases before producing a verdict. The idea extends retrieval-augmented generation [6] and the broader move toward language agents with long-term memory [7], [8], [9], [10].

Memory, however, is double-edged. The same mechanism that supplies useful prior signal can also propagate the judge's own past errors, can be deliberately corrupted [11], [12], and, most subtly, can leak information between related evaluation items. Benchmark contamination and evaluation leakage are recognized threats to the validity of LLM evaluation [13], [14], [15], and a memory that is updated during evaluation is a direct channel for such leakage. The topic of this thesis is therefore timely: as memory-augmented judges move toward practical use, the field needs a disciplined way to measure whether memory genuinely helps, and a way to detect when an apparent improvement is in fact an artifact of a leaky measurement.

1.2. Problem Statement

Given an LLM-as-a-Judge system, it is not currently straightforward to determine whether attaching a persistent memory improves the accuracy of its verdicts. A naive evaluation, which builds memory and tests on overlapping or paired data, or updates memory during testing, can report a large apparent gain that is caused by leakage rather than by genuine reasoning improvement. Conversely, a poorly instrumented evaluation harness can report large apparent failures that are caused by unhandled engineering faults rather than by the system under test. The problem is therefore twofold: to design an evaluation protocol that is provably free of memory leakage, and to use that protocol to obtain an honest, statistically grounded answer to the question of whether, when, and why memory helps.

1.3. Purpose and Research Question

The purpose of this work is to design, implement, and rigorously evaluate a memory-augmented LLM-as-a-Judge system, and to determine under what conditions persistent memory genuinely improves evaluation quality.

The thesis is organized around a single research question:

When does persistent memory help an LLM-as-a-Judge, when does it fail or corrupt the evaluation, and how can a memory-augmented judge be evaluated?

1.4. Objectives

To answer the research question, the following objectives were set:

1. Design and implement a memory-augmented judge (MAJ) that stores past evaluations in a structured, typed graph and retrieves relevant prior cases at evaluation time.
2. Implement two exploratory extensions that apply Monte Carlo Tree Search at the reasoning layer and at the memory-retrieval layer.
3. Design a leakage-free evaluation protocol that partitions the benchmark by question and freezes memory during testing.
4. Implement a two-layer frozen-memory audit that proves, for every run, that memory was not modified during evaluation.
5. Conduct controlled experiments comparing no-memory, self-written-memory, oracle-memory, and poisoned-memory conditions, with confidence intervals and paired significance tests.
6. Determine whether the apparent effect of memory survives a leakage-free, audited protocol, and explain the result mechanistically.

1.5. Object and Subject of Research

The **object** of research is the LLM-as-a-Judge evaluation process: the procedure by which a language model assigns a pass/fail verdict to a candidate response under a grading rubric. The **subject** of research is the effect of a persistent, retrieval-conditioned memory on the accuracy and the measurement

validity of that process, together with the protocol required to measure that effect without leakage.

1.6. Scientific Novelty

The scientific novelty of this thesis consists of the following.

1. **MCTS-MAJ as a research platform.** The thesis designs and implements MCTS-MAJ, a memory-augmented LLM-as-a-Judge built on a typed graph of past evaluations with a three-stage retrieval pipeline and two Monte Carlo Tree Search components. While memory-augmented and tree-search judging build on existing directions, the integrated system serves as the controlled platform on which the leakage question below is studied; it is, to date, the first memory-augmented judge whose evaluation is explicitly designed to be audited for leakage.

2. **The first provably leakage-free, audited evaluation of a memory-augmented LLM judge.** Question-level data splitting and state hashing are individually established techniques; the novelty here is their combination into a protocol applied, for the first time, to memory-augmented LLM-as-a-Judge systems, and made *provable* rather than assumed. The two-layer frozen-memory audit (a deterministic before-and-after snapshot with a SHA-256 fingerprint over the graph contents, together with runtime interception of every memory-write operation) is the first known evaluation harness for memory-augmented judging that proves, rather than assumes, the absence of memory leakage.

3. **A corrective empirical finding.** Applying the audited protocol overturns a previously plausible result: the large apparent benefit of memory observed under a conventional protocol does not survive, and on the studied benchmark no memory-augmented or tree-search configuration significantly

outperforms a single-pass stateless judge. The thesis further explains this finding mechanistically.

4. **A methodological demonstration.** When the measurement harness is not audited, it can surface effects that are not real. In this work these were a spurious accuracy gain and a spurious collapse under poisoning, and once the harness is audited and its error handling corrected, neither effect remains. The audited protocol is therefore shown to be a prerequisite for any trustworthy claim about memory-augmented judging."

1.7. Practical Significance

The leakage-free protocol and the frozen-memory audit form a small, self-contained instrument that any practitioner building a memory-augmented LLM judge can apply to obtain trustworthy measurements. The empirical finding is a cautionary result for practitioners: an apparent improvement from memory should not be believed until the evaluation has been audited for leakage and instrumented for transient failures. The complete implementation, including the audit, the statistical pipeline, and a one-command reproduction script, is released so that the protocol can be adopted directly.

1.8. Structure of the Thesis

Chapter 2 reviews the literature on LLM-as-a-Judge, retrieval- and memory-augmented systems, Monte Carlo Tree Search, evaluation validity, and statistical methods. Chapter 3 presents the methodology: the MCTS-MAJ architecture, the leakage-free protocol, and the frozen-memory audit. Chapter 4 describes the implementation. Chapter 5 reports and discusses the experimental results, organized around when memory appears to help, when it fails or misleads, and how it is audited. Chapter 6 concludes the thesis and outlines future work.

Chapter 2

2. Literature Review

2.1. LLM-as-a-Judge Evaluation

LLM-as-a-Judge refers to the use of a large language model to assess the quality of a response produced by another model or by a human [1], [2]. The judge is given a task description, a candidate response, and a grading rubric, and it returns a verdict, usually with a short justification. The appeal of the paradigm is scale: an LLM judge can score thousands of items at a fraction of the cost and latency of human annotation, and systems such as G-Eval [2] and Prometheus [16] report high agreement with human raters on several tasks.

The paradigm also has well-documented weaknesses. LLM judges exhibit systematic biases: they can be sensitive to the position of a response in a pairwise comparison, to response length, to self-preference, and to superficial stylistic features [17], [18]. Recent surveys of LLM-as-a-Judge reliability [4], [5] emphasize that automatic judges must themselves be evaluated carefully, that headline agreement numbers are often fragile, and that judge verdicts can be poorly calibrated [19]. A stateless judge, in particular, has no mechanism to be consistent

across related items, because it does not remember how it judged a similar case a moment earlier. This limitation is the entry point for memory augmentation.

2.2. Retrieval-Augmented and Memory-Augmented Systems

Retrieval-Augmented Generation (RAG) [6] extends a language model with an external store of documents that can be queried at inference time, so that the model's output is conditioned on retrieved evidence rather than on parametric knowledge alone. Subsequent work made retrieval adaptive: Active RAG [20] reformulates the retrieval query as reasoning proceeds, and Self-RAG [21] lets the model critique and selectively use its retrieved context.

A related line of work gives language agents an explicit long-term memory. MemGPT [7] treats memory management as an operating-system-like problem. Generative Agents [8] maintain a memory stream of past observations and retrieve from it by recency, importance, and relevance. Reflexion [9] stores verbal self-feedback from past attempts so that an agent can improve on later ones. Recent work specifically examines the benefits and risks of persistent memory in language-model agents [10], including the risk that stored content becomes stale, wrong, or adversarially corrupted [12].

Applying this idea to judging, a memory-augmented judge retrieves *past evaluations* rather than documents or episodes, and uses them as contrastive or confirmatory references. The present work differs from prior memory systems in two respects: the memory is organized as a *typed graph* rather than a flat store, enabling structured multi-hop retrieval; and the memory is studied specifically as a potential source of *evaluation leakage*, which prior memory-augmented work does not address.

2.3. Monte Carlo Tree Search and Test-Time Reasoning

Monte Carlo Tree Search (MCTS) is a heuristic search algorithm that incrementally builds a decision tree through repeated simulation [22], [23]. Each

iteration performs four steps (selection, expansion, simulation, and backpropagation), and the Upper Confidence Bound for Trees (UCT) rule balances exploitation of high-reward branches against exploration of less-visited ones. MCTS is best known for game playing, most famously as a component of AlphaGo [24].

Recent work adapts test-time search to LLM reasoning, a direction surveyed under the heading of test-time compute scaling [25]. Chain-of-Thought prompting [26] showed that decomposing a problem into intermediate steps improves accuracy; Self-Consistency [27] samples multiple reasoning paths and takes a majority vote; Tree of Thoughts [28] explores a tree of partial solutions; and ReAct [29] interleaves reasoning with actions. MCTS-Judge [30] applies MCTS directly to judging for code-correctness evaluation, treating each tree node as an evaluation subtask. The MCTS components in this thesis follow this line, but are reported as exploratory: as Chapter 5 shows, they do not yield a statistically significant accuracy gain on the studied benchmark.

2.4. Evaluation Validity, Leakage, and Contamination

Several recent studies converge on the same warning. A reported score can be pushed upward not by a better system but by contamination, that is, when the material used to construct a system also appears, directly or indirectly, in the data used to assess it. Surveys of data contamination [13] and studies of benchmark contamination [14], [15] show that test items appearing, directly or indirectly, in training or memory can substantially inflate reported scores. For retrieval systems specifically, knowledge-corruption attacks such as PoisonedRAG [11] demonstrate that injecting a small amount of false content into a retrieval store can change a system’s outputs.

For a memory-augmented judge, two leakage channels are particularly relevant. The first is *paired-item leakage*: many benchmarks contain closely related items, for example a passing and a failing answer to the same question, and if one

item is in memory while its partner is in the test set, the judge can effectively see the answer. The second is *write-back leakage*: if the memory is updated during evaluation, the verdict on an early test item becomes available when scoring a later, related one. The methodology of this thesis is designed to close both channels and, crucially, to *prove* that they are closed rather than merely assert it. This proof-oriented stance distinguishes the present work from prior memory-augmented evaluation, which typically assumes the absence of leakage.

2.5. Statistical Methods for Evaluation

Because LLM judges are stochastic and benchmarks are finite, point estimates of accuracy can be misleading. This thesis reports every accuracy with a two-sided Wilson confidence interval [31], which is well behaved for proportions at moderate sample sizes. Mode-versus-mode comparisons are made on paired data, the same test items judged by both modes, and assessed with the exact McNemar test [32], which conditions on the discordant pairs, together with a paired bootstrap confidence interval on the accuracy difference [33]. A difference is called statistically significant only when the McNemar p-value is below 0.05 and the bootstrap interval excludes zero. This conservative, paired methodology makes it possible to distinguish a genuine effect from sampling noise at the sample sizes used here. Recent commentary on machine-learning methodology argues that negative and null results, reported rigorously, are a legitimate and valuable scientific contribution [34]; this thesis is written in that spirit.

2.6. Summary

The literature establishes three points that frame this thesis. First, LLM-as-a-Judge is useful but fragile, and automatic judges must themselves be evaluated with care. Second, memory augmentation is a natural and active idea, but existing memory-augmented systems do not treat memory as a leakage risk. Third, contamination and leakage are known to inflate evaluation results, yet there is no

standard, proof-oriented protocol for evaluating a memory-augmented judge without leakage. The remainder of the thesis fills that gap.

Chapter 3

3. Methodology

3.1. Overview

This chapter presents the methodology used to design and evaluate the MCTS-MAJ system. It describes the typed memory graph and the memory-assisted judge, the two exploratory Monte Carlo Tree Search components, the leakage-free evaluation protocol, the frozen-memory audit, the memory conditions, and the statistical methods. It closes with the limitations and ethical considerations of the approach.

3.2. System Architecture

The MCTS-MAJ system has three components that operate on a shared memory graph: the Memory-Assisted Judge (MAJ), the MCTS-Judge, and the MCTS-Retrieval module. They combine into a small number of evaluation modes that serve as ablations.

3.2.1. The Typed Memory Graph

Memory is stored in a Neo4j graph database [35]. Unlike a flat document store, the graph uses five typed node kinds: **Policy** (a task or grading criterion), **Attempt** (a candidate response with its verdict and reasoning), **Issue** (a specific problem identified in an attempt), **Fix** (a corrective action that resolves an issue), and **Semantic** (an abstract category that groups similar issues). Typed relationships connect them: *SATISFIES* (Attempt to Policy), *CAUSES* (Attempt to Issue), *RESOLVES* (Fix to Issue), and *ABSTRACTS_TO* (Issue to Semantic). Every node carries a 1536-dimensional embedding of its text content, produced by the text-embedding-ada-002 model [36], and the graph maintains Hierarchical Navigable Small World vector indexes [37] for efficient similarity search.

3.2.2. The Memory-Assisted Judge

The Memory-Assisted Judge is a single-pass judge whose prompt is conditioned on retrieved memory. Retrieval proceeds in three stages: it finds the most similar past attempts, split into a positive and a negative set so that the judge sees a balanced contrastive signal; it retrieves issues semantically similar to the response under evaluation; and it aggregates recurring issue categories. Similarity thresholds are set asymmetrically (positive at least 0.85, negative at least 0.92) so that low-relevance items are dropped before they reach the judge. The retrieved material is formatted into a context block and supplied to the judge alongside the task and the response.

3.2.3. Exploratory MCTS Components

Two extensions apply Monte Carlo Tree Search and are reported as exploratory. **MCTS-Judge** decomposes an evaluation into five to seven task-specific subtasks generated by a planning step, and searches over reasoning trajectories using a UCT rule combined with an LLM self-assessment; the final verdict aggregates the best trajectory. **MCTS-Retrieval** replaces the fixed

three-stage retrieval with a tree search over seven retrieval actions, including four multi-hop graph traversals, scored by the relevance, diversity, and volume of the material retrieved. These components are described for completeness; Chapter 5 shows they do not produce a statistically significant accuracy gain, and the thesis does not claim a benefit for them.

3.2.4. Evaluation Modes

The components combine into the evaluation modes used as ablations: *Stateless* (single-pass judge, no memory); *MAJ* (single-pass judge with memory retrieval); *MCTS-Judge* (tree-search reasoning, no memory); and *MCTS-Judge + Memory* (tree-search reasoning with memory retrieval). Comparing these isolates the contribution of memory and of tree search.

3.3. The Leakage-Free Evaluation Protocol

3.3.1. The Leakage Problem

The benchmark used in this work, EvalsBench [38], contains paired items: for each question there is a passing and a failing response. This pairing creates two leakage channels. If the split is made at the level of individual rows, one version of a question may land in the memory-building set and the other in the test set, so the judge effectively sees the answer. If memory is updated during testing, the verdict on an early item becomes visible when a later, related item is scored. Either channel can inflate measured accuracy without any genuine improvement in judging.

3.3.2. Question-Level Partition

To close the first channel, the benchmark is split *by question*, not by row. The unique questions are shuffled with a fixed random seed and divided into a memory-building half and a test half; both the passing and the failing version of any given question always remain in the same half. No question, in any form, appears in both the memory and the test sets.

3.3.3. Frozen Memory During Evaluation

To close the second channel, the memory graph is *frozen* for the duration of test evaluation. The judge may read from memory, but no node or relationship is created, modified, or deleted while the test set is being scored. Memory is built once, before evaluation, and is read-only thereafter.

3.4. The Frozen-Memory Audit

A protocol that states memory is frozen is not enough; the absence of writes must be proven for every run. The frozen-memory audit provides this proof in two independent layers.

3.4.1. Layer One: Snapshot and Fingerprint

Immediately before and immediately after evaluation, a deterministic snapshot of the graph is taken. Each snapshot records the number of nodes of each type, the number of relationships of each type, and a SHA-256 fingerprint computed over the sorted list of node identifiers and edge triples. The two snapshots are compared: the run is reported as leakage-free only if the node and relationship counts are identical and the two fingerprints match exactly. The full before-and-after record is written to a per-run audit file, so the proof is preserved and independently checkable.

3.4.2. Layer Two: Runtime Write-Blocking

The snapshot check detects leakage after the fact. The second layer prevents it. For the duration of evaluation, every write method of the graph interface (node creation, relationship creation, get-or-create, and clear) is intercepted and replaced with a guard that raises a FrozenMemoryViolation exception if called. Any code path that attempts to write to memory during evaluation therefore fails immediately and loudly, rather than corrupting the run silently.

3.4.3. Value of the Audit

The audit was not a formality. When it was first applied to the MCTS-Judge + Memory mode, it failed: the evaluation path was writing the just-judged test item back into memory. This is exactly the write-back leakage the protocol is designed to exclude. The defect was found because the audit raised a violation, and it was fixed by adding a read-only evaluation mode. Every result reported in Chapter 5 has a passing audit record.

3.5. Memory Conditions

To study how the source and quality of memory affect the judge, four memory-building conditions are defined. In the **no-memory** condition the graph is empty and the judge is stateless. In the **self-written** condition the judge is run on the memory-building set and its own verdicts, reasoning, issues, and fixes are stored; this is how memory would grow in real deployment, and it includes the judge's own errors. In the **oracle** condition the ground-truth labels are stored directly without running the judge, giving a memory that is guaranteed correct. In the **poisoned** condition oracle memory is used but a fixed fraction of labels (10%, 20%, or 50%) is deterministically flipped, so the effect of corrupted memory can be measured.

3.6. Statistical Methodology

Every accuracy is reported with a two-sided Wilson 95% confidence interval [31]. Mode-versus-mode comparisons are computed on paired data, the identical test items judged by both modes, and assessed with the exact McNemar test [32] and a paired bootstrap 95% confidence interval with 10,000 resamples [33]. A difference is declared statistically significant only when the McNemar p-value is below 0.05 and the bootstrap interval excludes zero. This conservative rule guards against over-claiming at the sample sizes used. Where transient errors cause a

sample to fail after retries, that sample is excluded and the number of completed samples is reported alongside the accuracy.

It is important to state what effect size this design can detect. With eighty paired test items, a McNemar test at the 0.05 significance level reaches roughly eighty percent power only for a difference of about twelve to fifteen percentage points between two modes, given the discordant-pair rates observed in the experiments. The conventional, un-audited protocol suggested gains of six to ten percentage points; a gain of that magnitude, if it were genuine, would sit near or below this study's detection threshold. The null result reported in Chapter 5 should therefore be read precisely: the audited experiments were powered to detect a large memory effect and did not find one, but they cannot rule out a small genuine effect of a few percentage points. This is the reason the conclusions are framed as "no effect was detected at this scale" rather than as proof that no effect exists, and it is the reason the first item of future work is a substantially larger benchmark.

3.7. Limitations of the Methodology

Three limitations are stated in advance. First, the held-out test set contains eighty samples; a single misclassification shifts accuracy by 1.25 percentage points, so the study can detect only moderate-to-large effects, and a null result means “no effect was detected,” not “no effect exists.” Second, the audited experiments use a single judge model, GPT-4o; a weaker or a stronger model may behave differently. Third, the benchmark is a single domain, graded question answering, so the conclusions are stated for that setting.

3.8. Ethical Considerations

EvalsBench is a public-style benchmark with no personally identifying information, so no privacy concern arises from the data. The principal ethical commitment of the work is to measurement honesty: results are reported as they are, including the finding that the system does not produce a significant gain, and

including the discovery that an earlier reported effect was an artifact. All code, prompts, configurations, per-sample results, audit records, and a one-command reproduction script are released so that every number in the thesis can be independently checked.

Chapter 4

4. Implementation

4.1. Codebase

The system is implemented in Python. The module models.py defines the typed node data classes and an embedding helper; graph_manager.py manages the Neo4j connection, the vector indexes, the typed create and link operations, and the audit primitives; judge.py implements the single-pass judge used both statelessly and with memory; mcts_judge.py and mcts_retrieval.py implement the two exploratory search components; and mcts_pipeline.py composes the evaluation modes. The driver benchmark_leakage_free.py runs the leakage-free benchmark, including the audit wrapper and the transient-error handling. The analysis scripts analyze_stats.py, make_figures.py, and reproduce_all.py regenerate every statistic and figure and verify every artifact.

4.2. The Memory-Assisted Judge

The Memory-Assisted Judge (MAJ) is a single-pass judge whose prompt is conditioned on retrieved memory. Retrieval proceeds in three stages, shown in Listing 4.1: contrastive past attempts, semantically similar issues, and recurring semantic patterns. Each stage queries the typed graph through a top-k similarity

search; the retrieved material is then formatted into a single context block and supplied to the judge alongside the task and the candidate response.

```
def judge_with_memory(task, agent_output, graph_manager,
                      goal=None, model="gpt-4o"):
    code_embedding = get_embedding(agent_output)
    # Stage 1-2: contrastive attempts and similar issues
    contrastive = graph_manager.find_contrastive_attempts(
        code_embedding, top_k=3)
    similar_issues = graph_manager.find_similar_issues(
        code_embedding, top_k=5)
    # Stage 3: semantic patterns from the similar issues
    semantic_patterns = graph_manager.find_semantic_patterns(
        code_embedding, top_k=3)
    memory_context = _format_memory_context(
        contrastive, similar_issues, semantic_patterns)
    # ... judge call conditioned on memory_context ...
```

Listing 4.1 – The three-stage memory retrieval of the MAJ judge.

4.3. The MCTS Components

The MCTS-Judge explores alternative reasoning trajectories over a tree of evaluation subtasks. At each step it selects the child node with the highest Upper Confidence Bound for Trees (UCT) score, which balances exploitation of high-reward branches against exploration of less-visited ones; the implementation is shown in Listing 4.2.

```
def uct_score(self, exploration_constant=3.0):
    """Upper Confidence Bound for Trees."""
    if self.visit_count == 0:
        return float('inf')
    parent_visits = self.parent.visit_count if self.parent else 1
    exploitation = self.q_value
    exploration = exploration_constant * math.sqrt(
        math.log(parent_visits) / self.visit_count)
    return exploitation + exploration
```

Listing 4.2 – The UCT selection score used by the MCTS-Judge.

The MCTS-Retrieval component replaces the fixed three-stage pipeline with a tree search over a set of retrieval actions, including four multi-hop graph traversals.

The action set is defined declaratively, as shown in Listing 4.3, and each rollout assembles a trajectory of actions scored by the relevance, diversity, and volume of the material it retrieves. Both MCTS components are reported as exploratory; Chapter 5 shows they do not yield a statistically significant accuracy gain.

```
RETRIEVAL_ACTIONS = [
    {"name": "contrastive_attempts", ...},
    {"name": "similar_issues", ...},
    {"name": "semantic_patterns", ...},
    {"name": "multi_hop_issues_to_fixes", ...},
    {"name": "multi_hop_semantic_to_issues", ...},
    {"name": "multi_hop_policy_to_attempts", ...},
    {"name": "multi_hop_attempt_to_semantic", ...},
]
```

Listing 4.3 – The seven retrieval actions of MCTS-Retrieval, four of which are multi-hop graph traversals.

4.4. The Frozen-Memory Audit

The frozen-memory audit is the central engineering contribution of the implementation. Its first layer takes a deterministic snapshot of the memory graph before and after each evaluation. The snapshot records per-label node counts, per-relationship-type counts, and a SHA-256 fingerprint over the sorted node identifiers and edge triples, as shown in Listing 4.4. Two snapshots taken around a frozen-memory evaluation must be byte-identical; any divergence is direct evidence of a write that violates the leakage-free protocol.

```
def snapshot(self) -> dict:
    """Deterministic fingerprint of the current memory state."""
    import hashlib
    # ... collect node_counts, edge_counts, node_ids, edge_keys ...
    h = hashlib.sha256()
    h.update("|".join(node_ids).encode())
    h.update(b"\n")
    h.update("|".join(edge_keys).encode())
    return {
        "node_counts":  node_counts,
        "edge_counts":  edge_counts,
        "total_nodes":  sum(node_counts.values()),
        "total_edges":  sum(edge_counts.values()),
        "fingerprint":  h.hexdigest(),
    }
```

Listing 4.4 – The memory snapshot and SHA-256 fingerprint.

The second layer prevents leakage rather than only detecting it. For the duration of an evaluation, every write method of the graph interface is intercepted by a context manager that raises a FrozenMemoryViolation exception if any write is attempted. The audit is therefore defence-in-depth: the snapshot proves after the fact that nothing changed, and the write-blocker guarantees during the run that nothing can.

4.5. Structured Outputs

An early version of the MCTS-Judge extracted decisions with regular expressions over free-text model output, which failed silently on roughly 15 to 20 percent of responses that did not follow the expected template. All decision-extracting calls were migrated to a schema-validated structured-output interface; the schemas are shown in Listing 4.5. This eliminated parse failures without changing the semantics of any step.

```python
class SubtaskDecision(BaseModel):
    analysis: str
    decision: bool        # True = correct, False = incorrect

class SelfAssessment(BaseModel):
    useful: bool          # True = subtask improves evaluation

class GlobalVerdict(BaseModel):
    verdict: bool         # True = response passes
    reasoning: str
```

Listing 4.5 – Pydantic schemas for structured judge outputs.

4.6. Handling Transient Failures

LLM evaluation is network-bound. In an early run, a mid-evaluation network outage caused a block of samples to error, and the original code recorded each errored sample as an incorrect answer, silently depressing accuracy. Two corrections were made. An exponential-backoff retry wrapper, shown in Listing 4.6, was added around every model call. And the error handling was changed so that a sample that still fails after retries is recorded with a null verdict and excluded from the accuracy computation, rather than counted as wrong. This correction is material: it is the difference between an apparent catastrophic collapse under memory poisoning and the true, flat result reported in Chapter 5.

```python
def _call_with_retries(fn, *, max_attempts=5, base=2.0,
                       max_sleep=30.0):
    """Retry fn() on transient errors with backoff + jitter."""
    for attempt in range(1, max_attempts + 1):
        try:
            return fn()
        except Exception as exc:
            if not _is_transient(exc) or attempt == max_attempts:
                raise
            sleep_s = min(max_sleep, base ** attempt) \
                    + random.uniform(0, 0.5)
            time.sleep(sleep_s)
```

Listing 4.6 – Exponential-backoff retry wrapper for model calls.

4.7. Reproduction Pipeline

The benchmark driver builds memory for a chosen condition, runs the leakage-free evaluation with the frozen-memory audit enabled, and writes per-sample result files together with the audit records. The analysis script recomputes the Wilson intervals, the paired bootstrap intervals, and the exact McNemar tests from those per-sample files. The figure script regenerates every figure. A single reproduction script chains these steps, verifies that every expected result file and audit record is present, confirms that every audit passed, and exits with an error if any artifact is missing or any audit failed. Running that one script regenerates the entire empirical content of Chapter 5 from the raw per-sample data.

Chapter 5

5. Evaluation and Discussion

5.1. Dataset

All experiments use EvalsBench, an open benchmark released by Vibrant Labs AI under the Apache-2.0 licence, designed specifically to evaluate how well a Large Language Model performs as a judge of question-answer pairs. The benchmark is distributed as a single comma-separated file, benchmark_df.csv, and contains no personally identifying information.

The dataset comprises 160 samples covering 80 distinct topics in the startup and financial-analysis domain, for example pre-money valuation techniques, discounted-cash-flow analysis, and comparable-company analysis. Each row has six fields: the topic, the question, a set of grading notes that act as the rubric, the ground-truth target label (pass or fail), the candidate response under evaluation, and an optional notes field. The grading notes mark which points are critical and which are merely important, giving the judge a structured rubric to reason against.

Two properties of EvalsBench are central to this thesis. First, the target distribution is exactly balanced: 80 passing and 80 failing responses, so accuracy is

not inflated by a skewed prior. Second, and more importantly for the methodology, the benchmark is built around paired items. The 160 samples correspond to 80 unique questions, and each question appears twice, once with a passing response and once with a failing one. This pairing is what makes EvalsBench a natural stress test for evaluation leakage: if a passing and a failing version of the same question are split across the memory and the test sets, a memory-augmented judge can effectively retrieve the answer to the item it is scoring. The leakage-free protocol of Chapter 3 is designed precisely to neutralise this risk.

EvalsBench was chosen for three reasons. First, it is purpose-built for the LLM-as-a-Judge setting: every sample is a question, a rubric, and a candidate response with a ground-truth pass/fail label, which is exactly the input a judge must score, so no adaptation of the task is required. Second, its paired pass/fail design makes it an unusually direct test of evaluation leakage, the central methodological concern of this thesis; a benchmark without paired items would not expose the leakage channel as clearly. Third, it is openly released under a permissive licence, so every result reported here can be reproduced from the same public data.

5.2. Experimental Setup

Under the leakage-free protocol the benchmark is split by question into 40 memory-building questions (80 samples) and 40 test questions (80 samples), with the random seed fixed at 42 unless stated otherwise. The judge model is GPT-4o. The MCTS modes use two rollouts and a maximum depth of three; an earlier sweep to four rollouts and depth five gave no accuracy benefit at roughly four times the cost.

5.3. When Memory Appears to Help: The Conventional Protocol

Before the leakage-free protocol was designed, the MAJ core was characterized under a conventional protocol in which memory could be updated

during evaluation and the paired questions were not separated. Under that protocol memory appeared to help substantially: on a curated set of structured tasks the memory-assisted judge appeared to lift accuracy from 90% to 100%; across judge models, memory appeared to add six to ten percentage points for capable models while adding nothing for a weak one; and seeding the memory with 100 ground-truth examples appeared to raise cold-start accuracy from 78% to 91%.

These observations were obtained before the leakage-free protocol existed, and the per-sample logs for them were not retained; they are reported here as the *motivating observation* of the thesis rather than as verified claims. Several of the conventional-protocol configurations either update memory during evaluation or do not separate paired questions, and each of those is a leakage channel. The conventional protocol therefore establishes only that, under a naive evaluation, memory *appears* to help. The decisive question, whether the gain survives a leakage-free, audited protocol, is answered in the remainder of this chapter.

5.4. When Memory Fails to Help: The Leakage-Free Results

All results in this section use GPT-4o on 80 held-out samples with the memory graph frozen and the audit passing. Accuracies are reported with Wilson 95% confidence intervals; comparisons are paired, with McNemar tests and bootstrap intervals.

5.4.1. Clean-Memory Comparison

Table 5.1 *– Clean-memory results: leakage-free, audited, GPT-4o.*

Mode	Accuracy [95% CI]	n	Mean latency
Stateless	70.0% [59.2, 78.9]	80	3.6 s
MAJ (self-written memory)	65.0% [54.1, 74.5]	80	4.5 s
MCTS-Judge (no memory)	70.0% [59.2, 78.9]	80	31.8 s
MCTS-Judge + Memory	71.8% [61.0, 80.6]	78	33.3 s

Table 5.1 shows the four modes under clean, self-written memory. The four confidence intervals overlap substantially. The paired comparisons confirm that no difference is significant: MAJ versus stateless is -5.0 points (bootstrap CI $[-12.5, +1.3]$, McNemar $p = 0.29$); MCTS-Judge versus stateless is exactly 0.0 points; MCTS-Judge + Memory versus stateless is $+1.8$ points (McNemar $p = 0.80$). The single-pass stateless judge matches or exceeds every memory-augmented and tree-search configuration within statistical confidence.

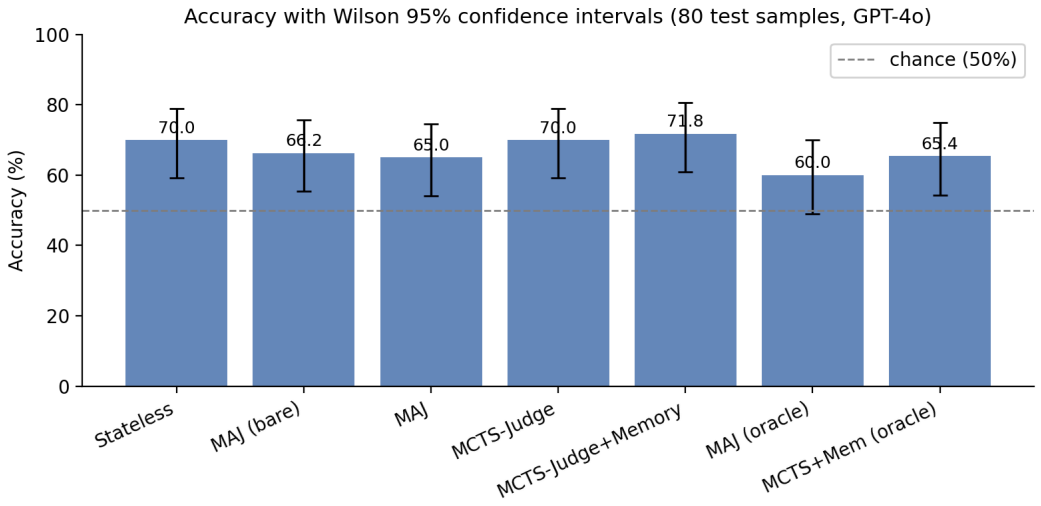

Figure 5.1 *– Per-mode accuracy with Wilson 95% confidence intervals on the leakage-free, audited test set. In addition to the four primary modes of Table 5.1, the figure also shows the bare-memory and oracle-memory variants discussed in Sections 5.4.2 and 5.4.3. The error bars of every mode overlap with those of the stateless baseline.*

Figure 5.1 presents the same data with confidence intervals drawn explicitly; the overlap of every interval with the stateless interval is the visual form of the central finding. Figure 5.2 shows the paired mode-versus-mode differences as a forest plot; every interval that involves a clean-memory comparison crosses zero.

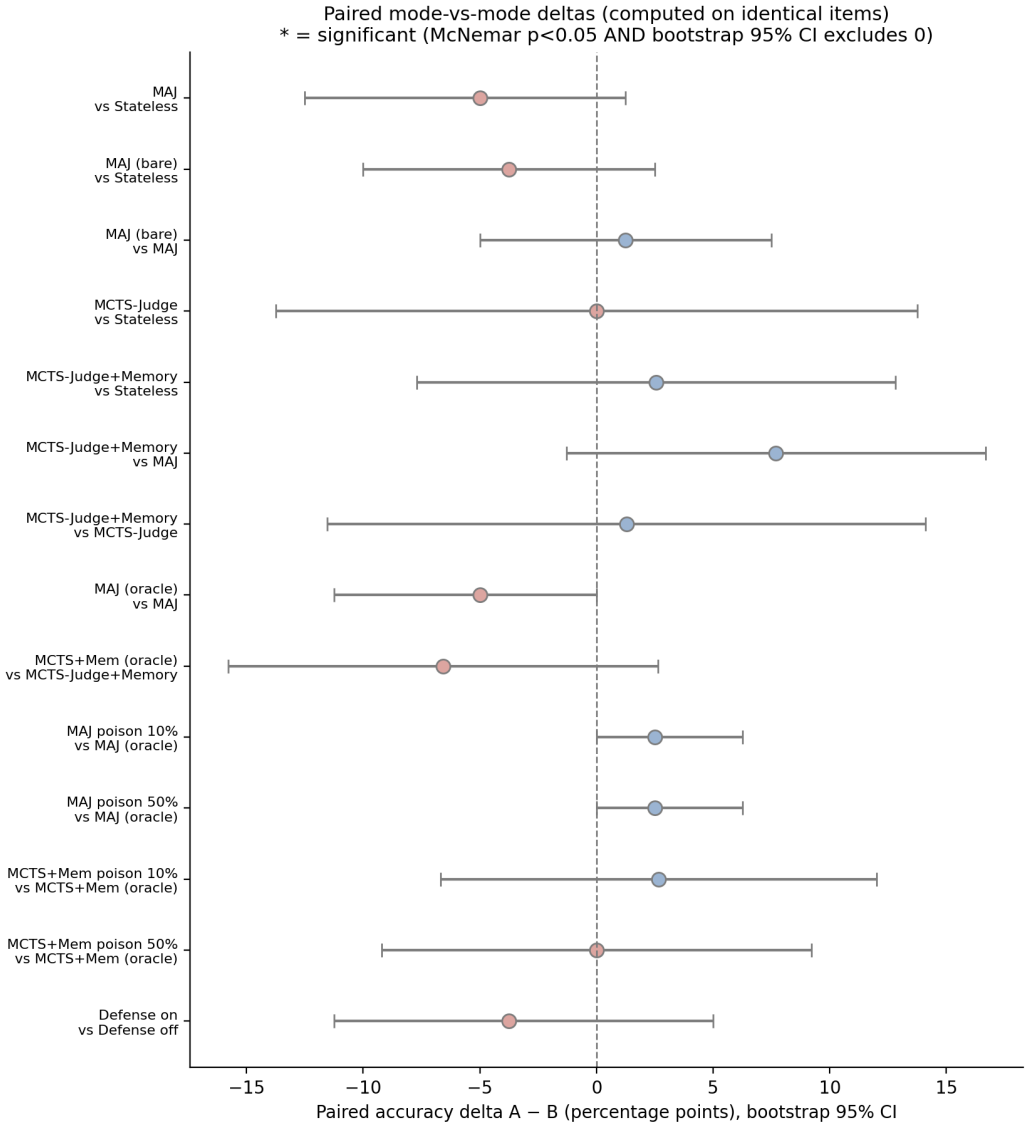

Figure 5.2 – *Paired accuracy differences between modes, computed on identical items, with bootstrap 95% confidence intervals. An interval crossing the dashed zero line indicates a non-significant difference; all clean-memory comparisons do so.*

This is the central result: under a leakage-free, audited protocol, the apparent benefit of memory observed under the conventional protocol does not appear.

A closer look at the per-mode error structure reinforces the point. Figure 5.3 shows the 2x2 confusion matrices for the four modes, and Figure 5.4 splits accuracy by ground-truth class. All four modes share the same characteristic error profile: they are markedly more accurate on passing responses than on failing ones,

a pass-oriented bias that memory and tree search do not correct. If memory were contributing genuine evaluative signal, it would be expected to shift this class-conditional error structure; it does not. The error profile is a property of the underlying judge model, not of whether memory is attached.

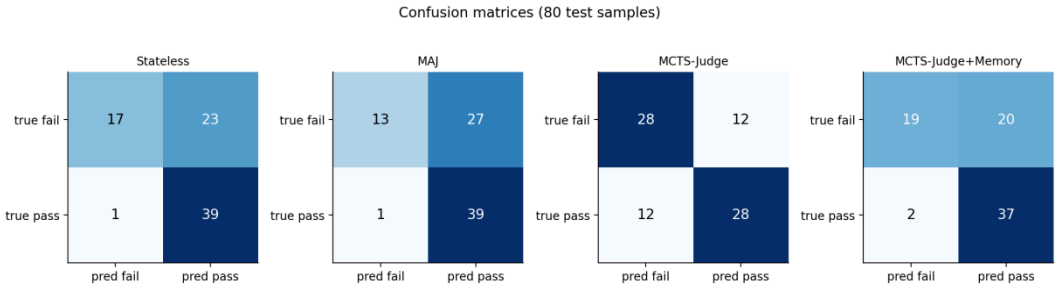

Figure 5.3 *– Per-mode confusion matrices on the leakage-free test set. The four modes share the same error structure, with more errors on failing responses than on passing ones.*

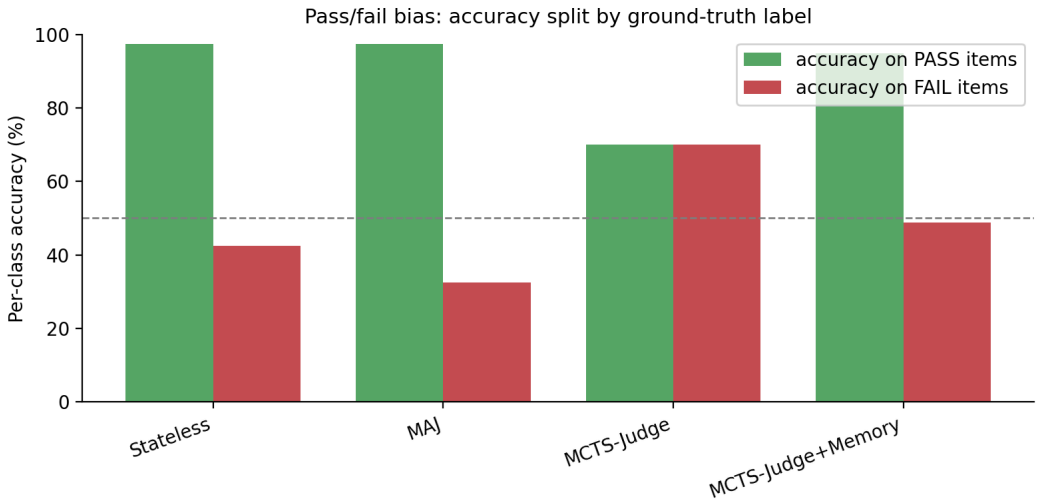

Figure 5.4 *– Per-class accuracy split by ground-truth label. Every mode is more accurate on passing responses than on failing ones; attaching memory does not correct this pass-oriented bias.*

5.4.2. Self-Written versus Oracle Memory

Table 5.2 *– Effect of memory provenance (leakage-free, audited).*

Memory provenance	MAJ	MCTS-Judge + Memory
Self-written	65.0%	71.8%
Oracle (ground-truth)	60.0%	65.4%

Replacing self-written labels with perfectly correct oracle labels does not improve either mode (Table 5.2); both are slightly lower with oracle memory, and neither difference is significant (MAJ $-\ 5.0$ points, $p = 0.22$; MCTS-Judge + Memory $-\ 6.6$ points, $p = 0.27$). This is itself informative. If the judge were genuinely exploiting the correctness of retrieved labels, perfectly correct memory should help. That it does not is consistent with the clean-memory finding that the memory channel carries little usable verdict signal on this benchmark.

5.4.3. Structure Ablation

To test whether the five-node typed graph earns its complexity, a “bare” memory was built using the same self-written labels but committing only Policy and Attempt nodes, with no Issue, Fix, or Semantic extraction. The bare memory scored 66.2% [55.4, 75.7], statistically indistinguishable from both the full five-node schema (65.0%) and the stateless baseline (70.0%).

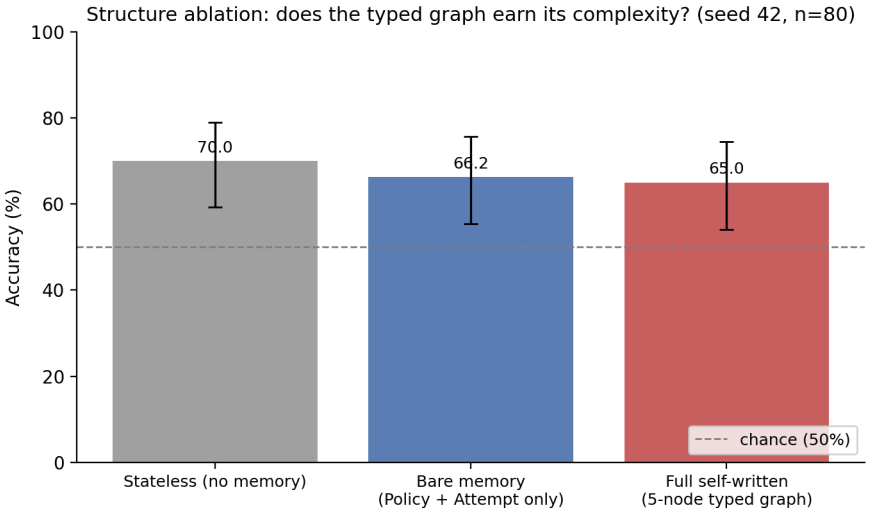

Figure 5.5 *– Structure ablation: accuracy of the stateless baseline, a bare Policy-plus-Attempt memory, and the full five-node typed graph, with Wilson 95% confidence intervals. The three intervals overlap.*

Figure 5.5 shows the three overlapping intervals. The additional extraction of Issue, Fix, and Semantic nodes does not produce a measurable verdict-accuracy gain on this benchmark.

5.4.4. Cross-Seed Stability

Table 5.3 *– Cross-seed comparison (leakage-free, audited).*

Mode	**Seed 42**	**Seed 123**	**Seed 7**
Stateless	70.0%	70.0%	67.5%
MAJ (self-written)	65.0%	67.5%	66.2%
MAJ (oracle)	60.0%	66.2%	66.2%

To confirm that the conclusion is not an artifact of a single split, the stateless and MAJ modes were re-run on two further random seeds, each with its own question-level split and a passing audit (Table 5.3).

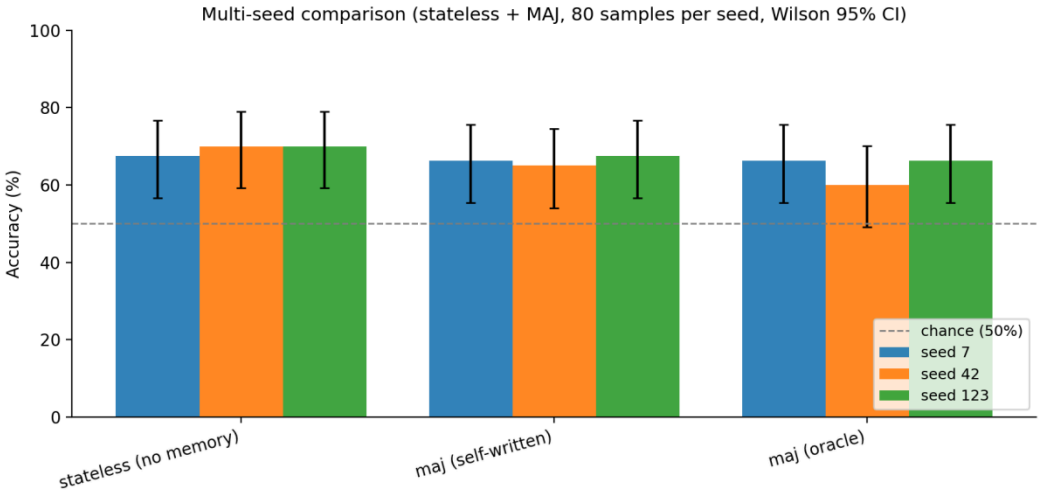

Figure 5.6 *– Cross-seed comparison of the stateless and memory-augmented modes over three random splits, with Wilson 95% confidence intervals. The cross-seed spread of each mode is wider than the gap between modes.*

As Figure 5.6 shows, across all three seeds MAJ tracks stateless within roughly two to five points and never significantly exceeds it. The per-mode range across seeds is about five points, wider than any mode-versus-mode difference on a single seed, which confirms both that the conclusion is stable and that small single-seed differences fall inside the cross-seed noise band.

5.5. When Memory Appears to Corrupt: Poisoning and the Artifact

Table 5.4 *– Robustness to label corruption (leakage-free, audited).*

Poison rate	MAJ	MCTS-Judge + Memory
0% (oracle)	60.0%	65.4%
10%	62.5%	67.5%
20%	63.7%	69.7%
50%	62.5%	65.4%

Under the audited protocol, neither mode degrades meaningfully as the poisoning rate rises (Table 5.4). The MAJ accuracy stays within a 60–64% band and the MCTS-Judge + Memory accuracy within a 65–70% band, across all poison

rates. For MCTS-Judge + Memory, 50% poisoning versus oracle is a paired difference of exactly 0.0 points (McNemar $p = 1.00$).

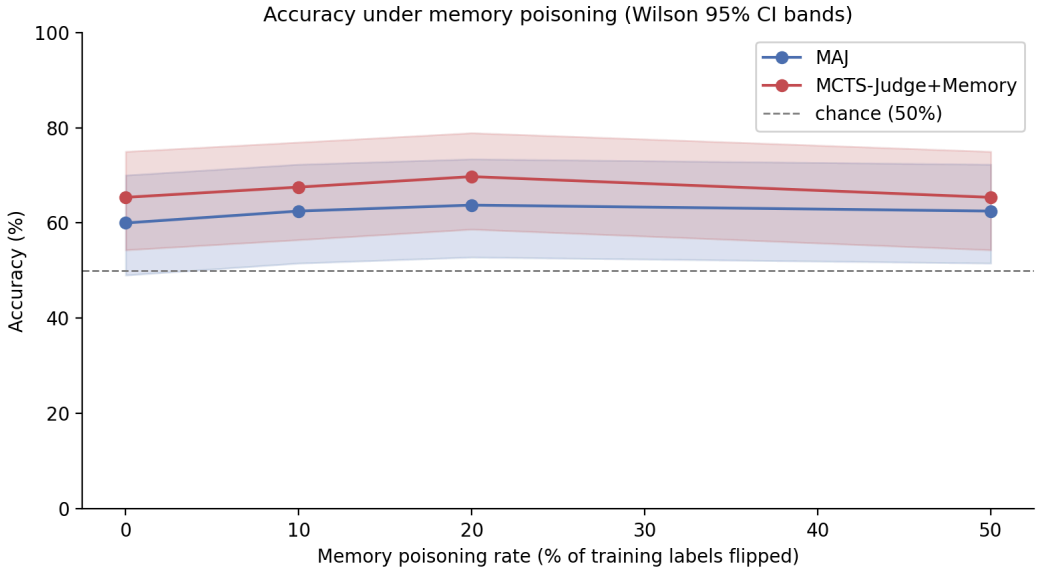

***Figure 5.7** – Accuracy as a function of the memory poisoning rate, under the corrected, audited harness, with Wilson 95% confidence interval bands. Neither mode shows a significant downward trend as the corruption rate rises.*

Figure 5.7 shows the flat poisoning curves. This corrects an earlier, un-audited version of the same experiment, which reported that MCTS-Judge + Memory collapsed to 21.2% accuracy at 50% poisoning, a 48.8-point drop. That earlier run predated the transient-error correction described in Chapter 4. On inspection of the per-sample logs, the collapse was an artifact: a mid-run network outage caused a block of samples to error, and the original code recorded each errored sample as a wrong answer. Once errored samples are retried and, on persistent failure, excluded rather than counted as wrong, the 50% poisoning condition scores 65.4%, identical to oracle. The dramatic "poisoning collapse" was a property of the measurement harness, not of the system.

5.6. Reliability Harness

Beyond raw accuracy, a reliability harness probes three further properties of the judge. *Stochastic stability* measures how often a mode returns the same verdict when the same item is judged twice. *Label-flip discrimination* measures how often a judge correctly changes its verdict when a response is altered from passing to failing. The *defense filter* tests whether a similarity-threshold filter on retrieved memory mitigates heavy poisoning. Figure 5.8 summarizes the results.

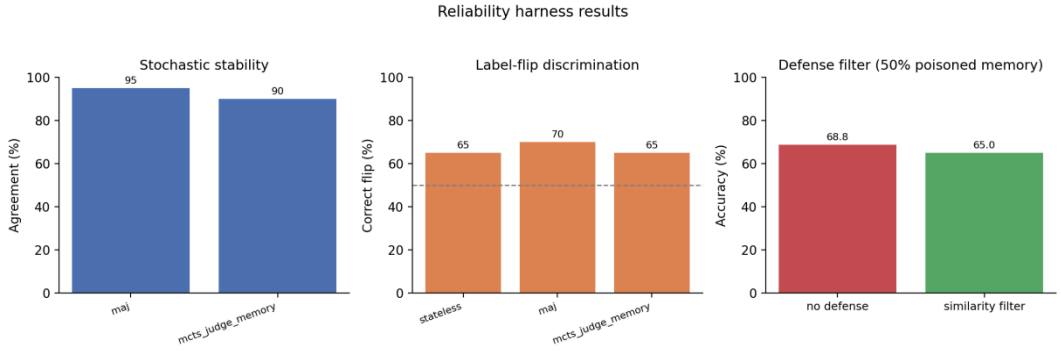

Figure 5.8 *– Reliability harness results: stochastic stability, label-flip discrimination, and the defense filter under 50% poisoned memory.*

Two findings are worth recording. The modes are highly stable across repeated runs, so the accuracy results above are not an artifact of stochastic noise within a single mode. The defense filter, however, does not help: under 50% poisoned memory the filtered configuration is no more accurate than the unfiltered one. Consistent with the rest of the chapter, this is reported as an honest negative result; a defense mechanism that does not measurably mitigate the failure mode it targets should be reported as such rather than presented as a working safeguard.

5.7. Discussion: Why Memory Neither Helped nor Harmed

The experimental program gives a clear, if unglamorous, answer to the research question, and this section explains the answer mechanistically, as required for a defensible scientific verdict.

Why the apparent benefit disappeared.

Under a conventional protocol, memory appeared to lift the judge by six to ten points; under the leakage-free, audited protocol, no lift is detectable. The difference between the two is the evaluation protocol, not the memory system. The most parsimonious explanation is that a large part of the conventional-protocol gain was an artifact of evaluation leakage: with paired pass/fail questions and memory updated during evaluation, the judge could effectively retrieve information about the very item it was scoring. Once the question-level split and the frozen-memory audit remove both leakage channels, that information is no longer available, and the apparent gain vanishes.

It should be stated plainly that this account is a hypothesis consistent with the evidence, not a directly tested result: the oracle-versus-self-written and bare-versus-full ablations are consistent with a judge operating near its reasoning ceiling, but the decisive test, a weaker judge model on which memory should help, is left to future work.

Why memory carries no usable signal here.

Three independent results point to the same mechanism. First, oracle memory, with perfectly correct labels, does not beat self-written memory (Table 5.2); if the judge were exploiting label correctness, perfect labels would help. Second, the bare Policy-plus-Attempt memory is indistinguishable from the full five-node schema (Figure 5.5); if structured issues and fixes carried signal, removing them would hurt. Third, the cross-seed range exceeds every mode-versus-mode gap (Figure 5.6); the differences that do appear are sampling noise. Together these indicate that GPT-4o, on graded question answering, is already operating near the ceiling its own reasoning permits: the task does not require knowledge that the model lacks, so a retrieved prior evaluation adds nothing the model could not already produce. Memory helps a judge only when it

supplies information the judge does not already have; on this benchmark, it does not.

Why the apparent collapse disappeared.

The earlier un-audited harness reported a catastrophic accuracy collapse under heavy poisoning. The corrected, retry-instrumented harness shows no significant degradation. The collapse was caused by transient API errors being recorded as wrong answers, not by corrupted memory. This is an engineering cause with a practical fix (retry plus correct error attribution), and once fixed, the effect does not exist.

When memory would be expected to help.

The null result is specific to this benchmark and judge, and the mechanism above predicts when the result would change. Memory should help when the judge cannot already solve the task from its own parametric knowledge: a weaker judge model, a harder or more specialised benchmark, or tasks whose correct verdict depends on domain knowledge or on conventions established only in the memory itself. It should also help when the same or near-identical items recur, so that a retrieved past verdict is directly reusable. These are testable predictions and are the natural next experiments; they could not be settled within the scope of this thesis, but the mechanistic account makes them concrete rather than speculative.

5.8. Cost of the MCTS Modes

The MCTS modes are roughly nine to thirteen times slower than the single-pass modes: a stateless evaluation averages 3.6 seconds per sample and MAJ 4.5 seconds, whereas MCTS-Judge averages 31.8 seconds and MCTS-Judge + Memory 33.3 seconds. Figure 5.9 plots accuracy against latency on a logarithmic scale, making the cost-reliability trade-off explicit: the MCTS modes sit an order of magnitude to the right of the single-pass modes without sitting any higher. Because the audited experiments find no significant accuracy advantage for

the MCTS modes, this additional cost is not currently justified by an accuracy return, and the MCTS extension is reported as exploratory.

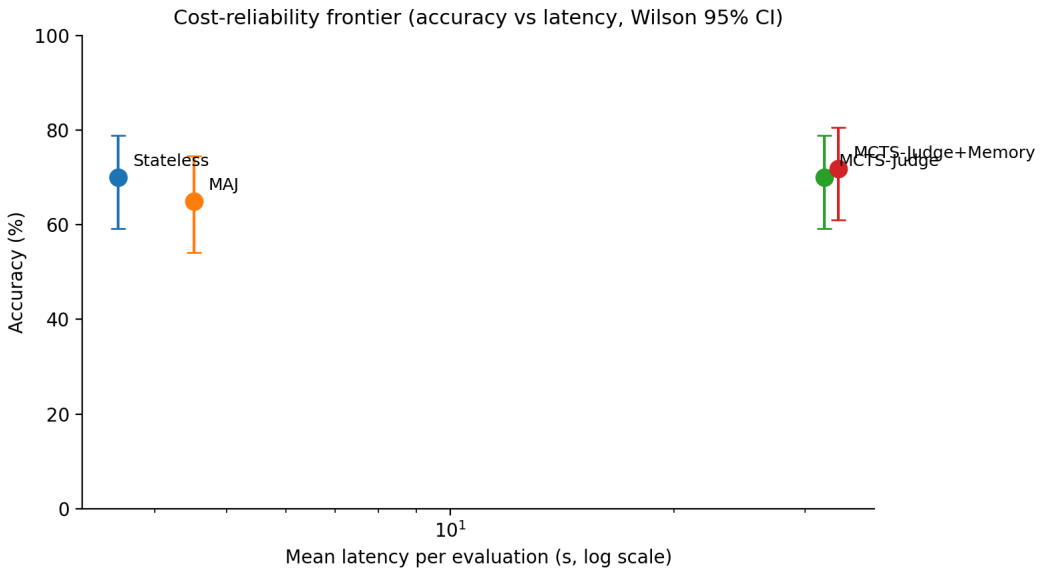

Figure 5.9 *– Cost-reliability frontier: accuracy against mean latency per evaluation (logarithmic scale), with Wilson 95% confidence intervals. The MCTS modes cost roughly an order of magnitude more latency without a corresponding accuracy gain.*

5.9. Threats to Validity

- **Sample size.** The held-out set has 80 samples; the conclusion is a null result, which at this sample size means no effect was detected, not that none exists. A larger benchmark could reveal a small genuine effect.
- **Single judge model.** The audited experiments use GPT-4o only; other models may behave differently, as the discussion above makes explicit.
- **Single benchmark.** EvalsBench is graded question answering; the conclusions are stated for that domain.
- **Single poisoning pattern.** Only uniform random label flipping was tested; structured adversarial poisoning may differ.

- **Conventional-protocol evidence.** The conventional-protocol per-sample logs were not retained, so those numbers are treated as a motivating observation rather than a verified claim.

Chapter 6

6. Conclusion

6.1. Summary of the Work

This thesis investigated whether a persistent, structured memory improves an LLM-as-a-Judge, and how such a system can be evaluated without leakage. A memory-augmented judging system, MCTS-MAJ, was designed and implemented, comprising a typed Neo4j memory graph, a retrieval-conditioned single-pass judge, and two exploratory Monte Carlo Tree Search components. A leakage-free evaluation protocol was designed, splitting the benchmark by question and freezing memory during testing, and a two-layer frozen-memory audit was implemented to prove, for every run, that memory was not modified during evaluation.

6.2. Answer to the Research Question

The research question asked when persistent memory helps an LLM judge, when it fails or corrupts the evaluation, and how a memory-augmented judge can be evaluated.

When memory appears to help. Under a conventional protocol, memory appeared to deliver large gains. Under the leakage-free, audited protocol, those

gains do not survive: on the studied benchmark no memory-augmented or tree-search configuration significantly outperforms a single-pass stateless judge, and this holds across three random seeds. The apparent benefit of memory was, to a large extent, an artifact of evaluation leakage.

When memory fails to help, and why. Memory does not help when the judge can already solve the task from its own reasoning. Oracle memory does not beat self-written memory, the bare graph does not lose to the full schema, and the differences that appear are within cross-seed noise. Together these show that, for GPT-4o on graded question answering, the memory channel carries no verdict signal the model did not already have.

When memory appears to corrupt, and why that is misleading. An un-audited harness reported a catastrophic accuracy collapse under heavy memory poisoning. Under a corrected, retry-instrumented harness, no significant degradation occurs; the collapse was an artifact of mishandled transient errors.

How to evaluate a memory-augmented judge without leakage. The contribution that holds is methodological: the question-level partition, the frozen memory, and the two-layer frozen-memory audit together form a protocol that proves the absence of leakage rather than assuming it. During development this audit caught a genuine leakage defect, which demonstrates its practical value.

The overall conclusion is that, on this benchmark, both the apparent benefit and the apparent fragility of memory in LLM-as-a-Judge evaluation are largely artifacts of the evaluation protocol, and that a leakage-free, audited protocol is a necessary precondition for any trustworthy claim about memory-augmented judging.

6.3. Contributions

The thesis makes the following contributions: the design and implementation of MCTS-MAJ, a memory-augmented LLM-as-a-Judge system built on a typed

Neo4j graph of past evaluations, with a three-stage retrieval pipeline and two exploratory Monte Carlo Tree Search components; a leakage-free evaluation protocol for memory-augmented LLM judges; a two-layer frozen-memory audit that proves the absence of memory leakage; an empirical finding, obtained under that protocol, that memory does not produce a statistically significant gain on the studied benchmark; a mechanistic explanation of that finding; and a methodological demonstration that an un-audited harness can report both gains and failures that do not exist.

6.4. Future Work

Several directions follow naturally. The benchmark should be enlarged well beyond 80 test samples, so that a small genuine effect, if one exists, could be resolved. The audited protocol should be applied to weaker and stronger judge models and to harder task domains, directly testing the prediction that memory helps when the judge cannot already solve the task. The leakage-free protocol and the frozen-memory audit could be packaged as a standalone, reusable evaluation tool. Finally, the structured memory graph, which did not improve verdict accuracy here, may still be valuable for interpretability and debugging, and that use deserves separate study.

6.5. Concluding Remark

The result of this thesis is a negative empirical finding paired with a positive methodological one. Memory did not measurably improve the judge; but the protocol built to test that question, and to catch the leakage and the artifacts that would otherwise have hidden the truth, is a reusable instrument, and the discipline it enforces is the lasting lesson of the work.

Appendix A. Frozen-Memory Audit Record Format

Each evaluation run produces a machine-readable audit record that proves the memory graph was not modified during evaluation. The record stores the before-and-after snapshots and their difference. A snapshot contains the per-type node counts, the per-type relationship counts, the total node and edge counts, and a SHA-256 fingerprint over the sorted node identifiers and edge triples. A run is reported as leakage-free only if the difference is empty and the two fingerprints are identical. For a passing run the difference flag is true, both delta maps are empty, and the before and after fingerprints match exactly. Every leakage-free experiment reported in Chapter 5 has a passing record of this form, and the records are released with the source code so that the absence of leakage can be checked independently.

Appendix B. Evaluation Modes and Memory Conditions

Table B.1 – *Evaluation modes.*

Mode	**Memory**	**Reasoning layer**
Stateless	none	single-pass judge
MAJ	three-stage retrieval	single-pass judge
MCTS-Judge	none	MCTS over subtasks
MCTS-Judge + Memory	three-stage retrieval	MCTS over subtasks

Table B.2 – *Memory-building conditions.*

Condition	What is stored
No memory	nothing; the graph is empty
Self-written	the judge’s own verdicts, reasoning, issues, and fixes
Oracle	the dataset ground-truth labels, stored directly
Poisoned (10/20/50%)	oracle memory with a fixed fraction of labels flipped

Appendix C. Full Per-Mode Results

Table C.1 collects every audited result in one place, with Wilson 95% confidence intervals and the number of completed samples. Where the completed count is below 80, the remaining samples errored after retries and were excluded from the accuracy computation, as described in Chapter 4.

Table C.1 – *Complete audited results (GPT-4o, seed 42).*

Configuration	**Accuracy [95% CI]**	**Completed** n
Stateless	70.0% [59.2, 78.9]	80
MAJ, self-written	65.0% [54.1, 74.5]	80
MAJ, oracle	60.0% [49.0, 70.0]	80
MAJ, poison 10%	62.5% [51.5, 72.3]	80
MAJ, poison 20%	63.7% [52.8, 73.4]	80
MAJ, poison 50%	62.5% [51.5, 72.3]	80
MCTS-Judge, no memory	70.0% [59.2, 78.9]	80
MCTS-Judge + Memory, self-written	71.8% [61.0, 80.6]	78
MCTS-Judge + Memory, oracle	65.4% [54.3, 75.0]	78
MCTS-Judge + Memory, poison 10%	67.5% [56.5, 76.9]	77
MCTS-Judge + Memory, poison 20%	69.7% [58.7, 78.9]	76
MCTS-Judge + Memory, poison 50%	65.4% [54.3, 75.0]	78

Appendix D. Reproduction

All numbers, tables, and figures in this thesis are reproducible from the released source code. A single reproduction script builds memory for each condition, runs the leakage-free evaluation with the frozen-memory audit enabled, recomputes the Wilson intervals, paired bootstrap intervals, and exact McNemar tests from the per-sample files, regenerates every figure, verifies that every expected artifact and audit record is present, confirms that every audit passed, and exits with an error if any artifact is missing or any audit failed.

Bibliography cited

[1] L. Zheng *et al.*, “Judging LLM-as-a-judge with MT-bench and chatbot arena,” in *NeurIPS datasets and benchmarks track*, 2023.

[2] Y. Liu, D. Iter, Y. Xu, S. Wang, R. Xu, and C. Zhu, “G-eval: NLG evaluation using GPT-4 with better human alignment,” *arXiv preprint arXiv:2303.16634*, 2023.

[3] J. Gu *et al.*, “A survey on LLM-as-a-judge,” *arXiv preprint arXiv:2411.15594*, 2024.

[4] H. Li, Q. Dong, J. Chen, H. Su, Y. Zhou, Q. Ai, Z. Ye, and Y. Liu, "LLMs-as-Judges: A Comprehensive Survey on LLM-based Evaluation Methods," arXiv preprint arXiv:2412.05579, 2024.

[5] A. S. Thakur, K. Choudhary, V. S. Ramayapally, S. Vaidyanathan, and D. Hupkes, "Judging the Judges: Evaluating Alignment and Vulnerabilities in LLMs-as-Judges," arXiv preprint arXiv:2406.12624, 2024.

[6] P. Lewis *et al.*, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” *NeurIPS*, 2020.

[7] C. Packer *et al.*, “MemGPT: Towards LLMs as operating systems,” *arXiv preprint arXiv:2310.08560*, 2023.

[8] J. S. Park, J. C. O’Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, “Generative agents: Interactive simulacra of human behavior,” *ACM Symposium on User Interface Software and Technology (UIST)*, 2023.

[9] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” *NeurIPS*, 2023.

[10] Z. Zhang, X. Bo, C. Ma, R. Li, X. Chen, Q. Dai, J. Zhu, Z. Dong, and J.-R. Wen, "A Survey on the Memory Mechanism of Large Language Model based Agents," arXiv preprint arXiv:2404.13501, 2024.

[11] W. Zou, R. Geng, B. Wang, and J. Jia, “PoisonedRAG: Knowledge corruption attacks to retrieval-augmented generation of large language models,” *USENIX Security Symposium*, 2024.

[12] L. Shi, C. Ma, W. Liang, X. Diao, W. Ma, and S. Vosoughi, "Judging the Judges: A Systematic Study of Position Bias in LLM-as-a-Judge," arXiv preprint arXiv:2406.07791, 2024.

[13] S. Yang, W.-L. Chiang, L. Zheng, J. E. Gonzalez, and I. Stoica, "Rethinking Benchmark and Contamination for Language Models with Rephrased Samples," arXiv preprint arXiv:2311.04850, 2023.

[14] C. Deng, Y. Zhao, X. Tang, M. Gerstein, and A. Cohan, “Investigating data contamination in modern benchmarks for large language models,” *Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2024.

[15] S. Balloccu, P. Schmidtova, M. Lango, and O. Dusek, "Leak, Cheat, Repeat: Data Contamination and Evaluation Malpractices in Closed-Source LLMs," in Conference of the European Chapter of the Association for Computational Linguistics (EACL), 2024.

[16] S. Kim *et al.*, “Prometheus: Inducing fine-grained evaluation capability in language models,” *arXiv preprint arXiv:2310.08491*, 2023.

[17] P. Wang *et al.*, “Large language models are not fair evaluators,” *arXiv preprint arXiv:2305.17926*, 2024.

[18] J. Ye *et al.*, “Justice or prejudice? Quantifying biases in LLM-as-a-judge,” *arXiv preprint arXiv:2410.02736*, 2024.

[19] R. Stureborg, D. Alikaniotis, and Y. Suhara, "Large Language Models are Inconsistent and Biased Evaluators," arXiv preprint arXiv:2405.01724, 2024.

[20] Z. Jiang *et al.*, “Active retrieval augmented generation,” *arXiv preprint arXiv:2311.11829*, 2023.

[21] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, “Self-RAG: Learning to retrieve, generate, and critique through self-reflection,” *arXiv preprint arXiv:2310.11511*, 2023.

[22] L. Kocsis and C. Szepesvari, “Bandit based monte-carlo planning,” *European Conference on Machine Learning (ECML)*, 2006.

[23] C. B. Browne *et al.*, “A survey of monte carlo tree search methods,” *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 4, no. 1, pp. 1–43, 2012.

[24] D. Silver, A. Huang, C. J. Maddison, *et al.*, “Mastering the game of go with deep neural networks and tree search,” in *Nature*, 2016, pp. 484–489.

[25] C. Snell, J. Lee, K. Xu, and A. Kumar, "Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters," arXiv preprint arXiv:2408.03314, 2024.

[26] J. Wei *et al.*, “Chain-of-thought prompting elicits reasoning in large language models,” *NeurIPS*, 2022.

[27] X. Wang *et al.*, “Self-consistency improves chain of thought reasoning in language models,” *International Conference on Learning Representations (ICLR)*, 2023.

[28] S. Yao *et al.*, “Tree of thoughts: Deliberate problem solving with large language models,” *NeurIPS*, 2023.

[29] S. Yao *et al.*, “ReAct: Synergizing reasoning and acting in language models,” *International Conference on Learning Representations (ICLR)*, 2023.

[30] Y. Wang, X. Chen, M. Zhao, and P. Liu, “MCTS-judge: Test-time scaling in LLM-as-a-judge for code correctness evaluation,” *arXiv preprint arXiv:2502.12468*, 2025.

[31] E. B. Wilson, “Probable inference, the law of succession, and statistical inference,” *Journal of the American Statistical Association*, vol. 22, pp. 209–212, 1927.

[32] Q. McNemar, “Note on the sampling error of the difference between correlated proportions or percentages,” *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.

[33] B. Efron and R. J. Tibshirani, *An introduction to the bootstrap*. Chapman; Hall/CRC, 1993.

[34] F. Karl, L. M. Kemeter, G. Dax, and P. Sierak, "Position: Embracing Negative Results in Machine Learning," arXiv preprint arXiv:2406.03980, 2024.

[35] Neo4j Inc., “Neo4j graph database: Vector search and graph indexes.” https://neo4j.com/docs/, 2023.

[36] OpenAI, “OpenAI text embeddings: Text-embedding-ada-002.” https://platform.openai.com/docs/guides/embeddings, 2024.

[37] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2018.

[38] Vibrant Labs AI, “EvalsBench: A benchmark for evaluating LLM-as-a-judge on graded question answering.” https://github.com/vibrantlabsai/EvalsBench, 2024.

[39] Pydantic Developers, “Pydantic: Data validation using python type hints.” https://docs.pydantic.dev/, 2024.